

NOMI DI ATTRIBUTI E RENAMING

- In SQL lo stesso nome può essere usato per più attributi solo se questi appartengono a relazioni diverse.
- Se una query coinvolge tali relazioni, occorre **qualificare** il nome dell'attributo con il nome della relazione per evitare ambiguità.
- ***Esempio:*** Employee.SSN

NOMI DI ATTRIBUTI E RENAMING (2)

- Si può avere ambiguità anche nel caso di query che riferiscono due volte alla stessa relazione:
 - **Esempio:** Per ogni impiegato, trovare il suo nome il suo cognome e quello del suo diretto superiore:

```
SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME  
FROM EMPLOYEE AS E, EMPLOYEE AS S  
WHERE E.SUPERSSN=S.SSN;
```
 - Abbiamo dichiarato nomi di relazione alternativi **E** ed **S**, detti **alias**, per la relazione EMPLOYEE .

NOMI DI ATTRIBUTI E RENAMING (3)

- È anche possibile rinominare gli attributi della relazione nella query, dando loro degli alias scrivendo:

- EMPLOYEE **AS** E(FN, MI, LN, SSN, BD, ADDR, SEX, SAL, SSSN, DNO)

nella clausola **FROM**.

- *Quella che abbiamo visto è un esempio di **query ricorsiva**.*

MANCANZA DEL WHERE

- Omettere la clausola WHERE equivale a **WHERE TRUE**, cioè tutte le tuple della relazione specificata nella clausola FROM fanno parte del risultato.
- Se più di una relazione è specificata nella clausola **FROM**, allora il risultato sarà il *prodotto cartesiano* delle relazioni.

MANCANZA DEL WHERE: ESEMPI

- **Esempio:** Selezionare tutti i SSN:

```
SELECT SSN  
FROM EMPLOYEE;
```

- **Esempio:** Selezionare tutte le combinazioni Employee.SSN e Department.Dname:

```
SELECT SSN, DNAME  
FROM EMPLOYEE, DEPARTMENT;
```

IL CARATTERE JOLLY “*”

- Per recuperare tutti gli attributi delle tuple selezionate, si usa il carattere jolly *:
- **Esempio:** Trovare tutti i valori degli attributi degli impiegati che lavorano per il dipartimento n°5:
SELECT *
FROM EMPLOYEE
WHERE DNO=5;
- **Esempio:** Trovare tutti gli attributi di Employee e gli attributi di Department per cui lavora ogni impiegato del dipartimento *‘Research’*:
SELECT *
FROM EMPLOYEE, DEPARTMENT
WHERE DNAME=‘Research’ **AND** DNO=DNUMBER;

DUPLICAZIONI DI TUPLE IN SQL

- SQL non tratta relazioni come insiemi: *tuple duplicate possono apparire più di una volta.*
- SQL non elimina le duplicazioni per le seguenti ragioni:
 - l'utente può essere interessato alle duplicazioni;
 - è un'operazione costosa (l'implementazione richiederebbe l'ordinamento e poi l'eliminazione);

LA CLAUSOLA DISTINCT

- Se le duplicazioni non sono volute, lo si specifica con la clausola **DISTINCT**:
- ***Esempi:***
 - Trovare i salari di tutti gli impiegati:
**SELECT SALARY
FROM EMPLOYEE;**
 - Trovare i salari distinti degli impiegati:
**SELECT DISTINCT SALARY
FROM EMPLOYEE;**

DISTINCT, *ATTENZIONE*

- cognome e filiale di tutti gli impiegati

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma

$\pi_{\langle \text{Cognome}, \text{Filiale} \rangle} (\text{Impiegati})$

SELECT

Cognome, Filiale

FROM Impiegati

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma
Rossi	Roma

SELECT DISTINCT

Cognome, Filiale

FROM Impiegati

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma

OPERAZIONI INSIEMISTICHE

- SQL incorpora le seguenti operazioni insiemistiche (è richiesta la union-compatibilità):

- UNION
- EXCEPT
- INTERSECT

} SQL 2

- **EXCEPT** restituisce tutti i valori distinti della query a sinistra dell'operando non presenti nella query a destra.
- Usando tali operazioni le tuple duplicate sono eliminate (a meno che non venga richiesto il contrario con la clausola **ALL**).

OPERAZIONI INSIEMISTICHE - *ESEMPIO*

- Fare una lista dei numeri di progetti per i progetti che coinvolgono un impiegato il cui cognome è *'Smith'* come lavoratore **oppure** come manager del dipartimento che controlla il progetto:

```
(SELECT PNUMBER
  FROM PROJECT, WORKS_ON, EMPLOYEE
  WHERE PNUMBER=PNO AND ESSN=SSN AND LNAME='Smith');
UNION
(SELECT PNUMBER
  FROM PROJECT, DEPARTMENT, EMPLOYEE
  WHERE DNUM=DNUMBER AND MGRSSN=SSN AND LNAME='Smith')
```

CONFRONTO TRA SOTTOSTRINGHE

- Per il confronto tra stringhe si usa l'operatore **LIKE**
- *Caratteri jolly:*
 - '%' rimpiazza qualsiasi numero di caratteri;
 - '_' rimpiazza un singolo carattere;
- **Esempio:** Trovare tutti gli impiegati il cui indirizzo è a *'Houston, Texas'*:

```
SELECT FNAME, LNAME  
FROM EMPLOYEE  
WHERE ADDRESS LIKE '%HOUSTON, TEXAS%';
```

“LIKE” - ESEMPIO

- Le persone che hanno un nome che inizia per 'A' e ha una 'd' come terza lettera:

```
SELECT *  
FROM persone  
WHERE nome LIKE 'A_d%';
```

CONFRONTO TRA SOTTOSTRINGHE - *ESEMPI*

- Trovare tutti gli impiegati nati negli anni '50. Il formato di data è *YYYY-MM-DD*:

```
SELECT FNAME, LNAME  
FROM EMPLOYEE  
WHERE BDATE LIKE '__ 5_____';
```

- Mostrare i salari risultanti se a tutti gli impiegati che lavorano sul progetto '*Product X*' fosse concesso un aumento del 10%:

```
SELECT FNAME, LNAME, 1.1*SALARY  
FROM EMPLOYEE, WORKS_ON, PROJECT  
WHERE ESSN=SSN AND PNO=PNUMBER AND PNAME='Product X';
```

GESTIONE DEI VALORI NULLI

Impiegati

Matricola	Cognome	Filiale	Età
5998	Neri	Milano	45
9553	Bruni	Milano	NULL

- Gli impiegati la cui età è NULL o potrebbe essere maggiore di 40.

σ **Età > 40 OR Età IS NULL (Impiegati)**

GESTIONE DEI VALORI NULLI (2)

- Gli impiegati la cui età è NULL o potrebbe essere maggiore di 40.

σ **Età > 40 OR Età IS NULL** **(Impiegati)**

```
SELECT *  
FROM Impiegati  
WHERE eta > 40 OR eta IS NULL
```